

Kotlin: Evolução da Programação Orientada a Objetos na JVM

Concisão, Segurança contra Nulos e Interoperabilidade Prática

André Wesley B. R. Filho

Leôncio Ferreira F. Neto

Paulo Gabriel L. Landim

Pedro Kauan C. da Silva

Ramona Vitória C. Cardoso

Salomão Rodrigues Silva

Prof. Rafael Will Macedo de Araújo — Paradigmas de Programação — UFCA

Kotlin vai substituir o Java?

O Problema Motivador no Java Clássico

Desafios da POO Tradicional no Java

- **Verbosidade Extrema:** Necessidade de escrever Getters, Setters, equals(), hashCode() e toString() manuais.
- **NullPointerException (NPE):** O erro de "1 bilhão de dólares" em tempo de execução.
- **Boilerplate Impeditivo:** Muito código de infraestrutura para pouca regra de negócio.

Sintoma Típico em Java

Uma simples classe de modelo com 2 atributos em Java costuma exigir mais de 40 linhas de código boilerplate apenas para funcionar adequadamente.

História do Kotlin, JVM & A Era Android

Timeline de Evolução e o Fator Google

- **2011:** JetBrains cria o Kotlin focado em produtividade.
- **Processo Oracle vs Google:** A disputa legal (iniciada pela Oracle/Sun) sobre patentes do Java no Android motivou a Google a buscar alternativas mais seguras.
- **2017:** Google adota o Kotlin como linguagem oficial para Android.
- **2019:** Google declara o ecossistema Android *Kotlin-First*.
- **Jetpack Compose:** Novo toolkit de UI, 100% em Kotlin.

Compilação na JVM

Código Kotlin (.kt) → Compilador `kotlinc` → Bytecode (.class) → JVM / ART

Aplicações e Principais Vantagens

Onde o Kotlin é Aplicado?

- **Mobile:** Linguagem oficial e recomendada pelo Google para *Android*.
- **Backend:** Suporte nativo e de primeira classe no *Spring Boot* e frameworks próprios como *Ktor*.
- **Multiplataforma (KMP):** Compartilhamento de lógica de negócio entre *iOS*, *Android*, Desktop e Web.
- **Data Science & Scripts:** Crescente adoção com ferramentas interativas e

Por que as empresas migram?

- **Concisão:** Reduz drasticamente o código *boilerplate* (código inútil que precisa ser escrito repetidamente).
- **Segurança:** Elimina quase que totalmente os erros de ponteiro nulo (o temido *NPE*).
- **Interoperabilidade:** Aproveita 100% do ecossistema e bibliotecas Java já existentes sem dor de cabeça.

Evolução de POO: Data Classes e Imutabilidade

Código do Nosso Projeto (Aluno.kt):

```
1  data class Aluno(  
2      val id: String,  
3      val nome: String,  
4  ) {  
5      init {  
6          require(id.isNotBlank()) {  
7              "O identificador do aluno nao pode  
8                  ↳ estar vaziao."  
9          }  
10         require(nome.isNotBlank()) {  
11             "O nome do aluno nao pode estar  
12                 ↳ vaziao."  
13         }  
14     }  
15 }
```

O que a data class gera

- equals() e hashCode()
- toString() formatado
- copy() para cópias

val vs var

val = imutável (padrão seguro)

var = mutável (uso explícito)

Null Safety Nativo no Sistema de Tipos

Controle de Nulos

- **Tipos Não-Anuláveis:** `val nome: String` nunca aceita `null`.
- **Tipos Anuláveis:** `val apelido: String?` permite nulo explicitamente.

Operadores Seguros

- **Safe Call** (`?.`)
- **Elvis** (`?:`)
- **Not-Null Assertion** (`!!`)

```
1 // Tipo anulavel
2 val apelido: String? = null
3
4 // Safe Call - retorna null sem erro
5 val tamanho = apelido?.length
6
7 // Elvis Operator - valor padrao
8 val exibe = apelido ?: "Sem apelido"
9
10 // Conversao segura
11 val texto: Any = "Kotlin"
12 val str = texto as? String
```

O Poder dos Smart Casts

O Problema no Java

Em Java, mesmo após verificar se um objeto é de um determinado tipo com `instanceof`, você ainda é obrigado a realizar o *cast* explícito e manual para usar os métodos.

A Solução Kotlin

O compilador rastreia a verificação feita com o operador `is`. Se for verdadeira, a variável sofre o **Smart Cast** (conversão automática) instantaneamente!

Comparação de Código:

```
1 // Em Java: Boilerplate de Cast
2 if (pessoa instanceof Aluno) {
3     Aluno aluno = (Aluno) pessoa;
4     aluno.fazerProva();
5 }
```

```
1 // Em Kotlin: Smart Cast Automatico
2 if (pessoa is Aluno) {
3     // Aqui 'pessoa' ja age como Aluno
4     pessoa.fazerProva()
5 }
```

Funções de Extensão & Expressividade

Código do Nosso Projeto (Extensoes.kt):

```
1 fun List<Avaliacao>.calcularMediaGeral(): Double {  
2     if (this.isEmpty()) return 0.0  
3     val somaDasMedias = this.sumOf { avaliacao ->  
4         CalculadoraDeMedia(avaliacao.notas[0], avaliacao.notas[1]).calcularMedia()  
5     }  
6     return somaDasMedias / this.size  
7 }
```

Poder das Extension Functions

- Adiciona métodos a classes existentes (ex: List) sem herança ou alteração do código-fonte.
- Uso expressivo: `avaliacoes.calcularMediaGeral()`

Interoperabilidade Java ↔ Kotlin

Java (GeradorDeMensagemJava.java):

```
1 public class GeradorDeMensagemJava {
2     private String remetente;
3
4     public GeradorDeMensagemJava(String
5         ↪ remetente) {
6         this.remetente = remetente;
7     }
8
9     public String gerarMensagemBoasVindas(
10         String nomeAluno) {
11         return "Ola " + nomeAluno
12             + ", bem-vindo ao sistema!";
13     }
14 }
```

Kotlin (GeradorDeMensagemKotlin.kt):

```
1 class GeradorDeMensagemKotlin(
2     private val remetente: String
3 ) {
4     fun gerarMensagemBoasVindas(nomeAluno:
5         ↪ String) =
6         "Ola $nomeAluno, bem-vindo ao
7         ↪ sistema!"
8 }
```

Concisão e Interoperabilidade

O que exige 13 linhas em Java é escrito em 6 linhas no Kotlin. E a interoperabilidade é de mão dupla: arquivos Java clássicos conseguem instanciar essa classe Kotlin nativamente!

POO Integrada à Programação Funcional

Recursos Funcionais

- Funções como cidadãs de primeira classe.
- Expressões Lambda e High-Order Functions.
- Manipulação declarativa de coleções.

Benefício Prático

Elimina loops imperativos manuais, aumentando legibilidade e reduzindo probabilidade de bugs.

Exemplo do Nosso Projeto:

```
1 // EstatisticaTurmaService.kt
2 val alunosAvaliados = todosAlunos.filter {
3     avaliacaoRepo.buscarPorIdAluno(it.id)
4         .isEmpty()
5 }
6
7 val mediaTurma = alunosAvaliados.map {
8     avaliacaoRepo.buscarPorIdAluno(it.id)
9         .calcularMediaGeral()
10 }.average()
```

Nosso Protótipo Prático: Gestão Acadêmica

Validação Prática dos Conceitos

Desenvolvemos uma aplicação acadêmica funcional em Kotlin para consolidar a interoperabilidade com Java e POO avançada.

Métricas do Código-Fonte

31 Classes • **7** Interfaces • **74** Testes Automatizados

Arquitetura em Camadas (`sistema.alunos`)

- **model**: Entidades puras usando `data class` e imutabilidade.
- **service / repository**: Regras de negócio, interfaces e *Extension Functions*.
- **interop**: Integração nativa com rotinas legadas em Java.
- **controller / view**: Fluxo de execução no console.

Panorama de Mercado Global

Adoção Corporativa

Grandes empresas globais que adotam Kotlin em escala de produção em seus principais aplicativos e serviços de backend:



Mercado de Trabalho no Brasil

Oportunidades Ativas

Forte adoção no setor financeiro, e-commerce e startups brasileiras. Mais de **2.000 vagas abertas** registradas no LinkedIn Brasil para profissionais Kotlin.

The screenshot shows a LinkedIn search results page for 'kotlin em: Brasil'. The top navigation bar includes filters for 'Vagas', 'Data do anúncio', 'Nível de experiência', 'Empresa', 'Remoto', and 'Todos os filtros'. The search results list several job openings:

- Arquiteto de Soluções Mobile Sênior** at Skopla Digital (Brazil, Remote). Promoted by recruiter. 46 people clicked on 'Candidate-se'.
- Desenvolvedor Fullstack Júnior | Foco em Inteligência Artificial** at Envolvora (Extrema, Minas Gerais, Brazil, Presencial). Promoted.
- Quality Assurance Engineer (São Paulo, BR)** at B Lab (São Paulo, Brazil, Hybrid). Promoted.
- Desenvolvedor(a) BackEnd** at Avenode (Recife, Pernambuco, Brazil, Presencial). Promoted. 1 ex-aluno trabalhou aqui.
- Estágio em Desenvolvimento Backend** at Bemobi Wave (Brazil, Remote). Há 2 dias.
- Estágio em Desenvolvimento (Javascript/Python)** at Luby (Brazil, Remote). Há 1 mês. Remoto.
- Software Development Intern** at Intech.

The detailed view of the 'Arquiteto de Soluções Mobile Sênior' job is shown on the right. It includes the company logo, job title, location, and a description: 'Estamos em busca de um(a) Arquiteto(a) Mobile para atuar como principal referência técnica na definição e evolução de arquitetura de uma organização do setor financeiro, garantindo coerência, escalabilidade e sustentabilidade das decisões técnicas ao longo do tempo. Essa posição terá papel estratégico na construção de soluções mobile robustas, escaláveis, seguras e de alta performance, estabelecendo padrões arquiteturais, apoiando os times de desenvolvimento e garantindo a evolução sustentável das aplicações. Buscamos um profissional com forte experiência em Flutter, visão arquitetural e capacidade de transitar entre decisões estratégicas e detalhes técnicos, incluindo interações nativas com Android e iOS'.

Como Começar no Kotlin

Trilha Recomendada

- 1 **Kotlin Playground:** Teste código diretamente no navegador (`play.kotlinlang.org`).
- 2 **Ambientes Integrados (IDE):** IntelliJ IDEA e Android Studio.
- 3 **Frameworks de Mercado:** Spring Boot ou Ktor.
- 4 **Kotlin Multiplatform (KMP):** Compartilhamento de código nativo.



Guia Completo

Acesse a nossa Wiki! Lá você encontra o **Guia de Instalação**, **Cursos Recomendados** e todos os

Limitações e Desafios Práticos

Compilação Mais Lenta

O rigor do compilador (*null safety* e inferência) torna o *build* mais pesado.

Na Prática: Nosso *pipeline* com **Gradle** demorou visivelmente mais para rodar do que em ecossistemas mais ágeis como o **Node.js**.

Adoção Fora da JVM

Apesar do *Multiplatform*, a comunidade e as bibliotecas fora do eixo Android/Backend ainda são muito menores que Python ou JavaScript.

Curva de Aprendizado e o Operador !!

- **Transição:** Leva tempo para parar de escrever “Java com sintaxe Kotlin” e usar o modo funcional.
- **Falsa Segurança:** Abusar do operador !! (forçar não-nulo) anula totalmente a proteção contra *NullPointerExceptions*.

A Grande Pergunta

NÃO.

Por que?

- O ecossistema Java é gigantesco, robusto e continua evoluindo (ex: *Records*, *Virtual Threads*).
- A interoperabilidade perfeita significa que não é necessário reescrever ou abandonar sistemas legados.
- O Kotlin atua como uma evolução complementar, entregando produtividade onde faz sentido, mas ambas coexistirão pacificamente na JVM.

Referências

Código-Fonte (GitHub)



[https://github.com/LeoncioFerreira/
Kotlin-Evolucao-da-Programacao-Orientada-a-Objetos-na-JVM](https://github.com/LeoncioFerreira/Kotlin-Evolucao-da-Programacao-Orientada-a-Objetos-na-JVM)

Guia de Instalação e Cursos



Escaneie para acessar a **Wiki** oficial do projeto com tutoriais, instalação e cursos sugeridos.

Referências Bibliográficas

As referências bibliográficas completas utilizadas para esta apresentação encontram-se detalhadas na **Wiki** do projeto.

Muito obrigado pela atenção!